Name: Asad
Class: BSAI 4C
Roll Number: 141

ARTIFICIAL INTELLIGENCE
LAB ASSIGNMENTS

Instructor: Rasikh Ali

Lab 8: Minimax Algorithm

In this lab, I learned how the minimax algorithm works for two-player games. The algorithm explores all possible moves assuming the opponent also plays optimally. I coded a recursive function that alternates between maximizing and minimizing players until the target depth is reached. The base case simply returns the score at the leaf node. Then I tested it with a small tree of scores.

```
import math

def minimax(curDepth, nodeIndex, maxTurn, scores, targetDepth):
    # base case : targetDepth reached
    if curDepth == targetDepth:
        return scores[nodeIndex]

    if maxTurn:
        return max(minimax(curDepth + 1, nodeIndex * 2, False, scores, targetD
epth),
                   minimax(curDepth + 1, nodeIndex * 2 + 1, False, scores, tar
getDepth))
    else:
        return min(minimax(curDepth + 1, nodeIndex * 2, True, scores, targetDe
pth),
                   minimax(curDepth + 1, nodeIndex * 2 + 1, True, scores, targ
etDepth))

# Driver code
scores = [3, 5, 2, 9, 3, 5, 2, 9]
treeDepth = math.log(len(scores), 2)
print("The optimal value is : ", end="")
print(minimax(0, 0, True, scores, treeDepth))
```

Lab 9: Data Loading & Exploration

For this lab, I picked a dataset from Kaggle and performed basic exploration using pandas. I read the file, displayed the first and last few rows, checked the shape, identified missing values, and filled those nulls using the mode.

```
import pandas as pd

# 1. Read CSV
dataset = pd.read_csv('filename.csv')

# 2. Top 5 rows
dataset.head()

# 3. Bottom 5 rows
dataset.tail()

# 4. Rows and columns
print('rows: ', dataset.shape[0])
print('columns: ', dataset.shape[1])

# 5. Null columns
dataset.isnull().sum()

# 6. Fill null values using mode
dataset = dataset.fillna(dataset.mode)

# 7. Datatypes of each column
dataset.dtypes
```

Lab 10: Data Pre-Processing

I continued cleaning the data: checked nulls, explored unique values, filled missing values with the mode, converted datatypes to int, dropped irrelevant columns, and encoded object columns using factorize.

```
import pandas as pd
import numpy as np

data = pd.read_csv('dataset.csv')

print('We have {} rows.'.format(data.shape[0]))
print('We have {} columns'.format(data.shape[1]))
print(np.sum(pd.isnull(data)))
print(data['columnname'].unique())

num = data['columnname'].mode()[0]
data['columnname'] = data['columnname'].fillna(num)

data.columnname = data.columnname.astype(np.int64)
data.drop('id', axis=1, inplace=True)
print(data.dtypes)

x = data.iloc[:, 0:-1]
y = data.iloc[:, -1]

cat_columns = x.select_dtypes(['object']).columns
x[cat_columns] = x[cat_columns].apply(lambda x: pd.factorize(x)[0])
```

Lab 11: Model Training & Evaluation

After preprocessing, I split the data, applied six classifiers, and evaluated them using accuracy, precision, recall, and F1-score. I also plotted comparison graphs.

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import BernoulliNB, GaussianNB, MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
import matplotlib.pyplot as plt
import numpy as np

X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.3, shuff
le=False)

# Bernoulli NB
BernNB = BernoulliNB()
BernNB.fit(X_train, Y_train)
Y_bpred = BernNB.predict(X_test)
b_accuracy = metrics.accuracy_score(Y_test, Y_bpred)
b_precision = metrics.precision_score(Y_test, Y_bpred, average='weighted', lab
els=np.unique(Y_bpred))
b_recall = metrics.recall_score(Y_test, Y_bpred, average='weighted', labels=np
.unique(Y_bpred))
b_f1 = metrics.f1_score(Y_test, Y_bpred, average='weighted', labels=np.unique(
Y_bpred))
print('Bernoulli - Acc:', b_accuracy, 'Prec:', b_precision, 'Rec:', b_recall,
'F1:', b_f1)

# Random Forest
RF = RandomForestClassifier()
RF.fit(X_train, Y_train)
Y_rpred = RF.predict(X_test)
r_accuracy = metrics.accuracy_score(Y_test, Y_rpred)
r_precision = metrics.precision_score(Y_test, Y_rpred, average='weighted', lab
els=np.unique(Y_rpred))
r_recall = metrics.recall_score(Y_test, Y_rpred, average='weighted', labels=np
.unique(Y_rpred))
r_f1 = metrics.f1_score(Y_test, Y_rpred, average='weighted', labels=np.unique(
Y_rpred))
print('Random Forest - Acc:', r_accuracy, 'Prec:', r_precision, 'Rec:', r_reca
ll, 'F1:', r_f1)

# Gaussian NB
GausNB = GaussianNB()
GausNB.fit(X_train, Y_train)
Y_gpred = GausNB.predict(X_test)
g_accuracy = metrics.accuracy_score(Y_test, Y_gpred)
g_precision = metrics.precision_score(Y_test, Y_gpred, average='weighted', lab
els=np.unique(Y_gpred))
g_recall = metrics.recall_score(Y_test, Y_gpred, average='weighted', labels=np
.unique(Y_gpred))
g_f1 = metrics.f1_score(Y_test, Y_gpred, average='weighted', labels=np.unique(
Y_gpred))
print('GaussianNB - Acc:', g_accuracy, 'Prec:', g_precision, 'Rec:', g_recall,
'F1:', g_f1)

# Decision Tree
Dtree = DecisionTreeClassifier()
Dtree.fit(X_train, Y_train)
Y_dpred = Dtree.predict(X_test)
d_accuracy = metrics.accuracy_score(Y_test, Y_dpred)
d_precision = metrics.precision_score(Y_test, Y_dpred, average='weighted', lab
els=np.unique(Y_dpred))
d_recall = metrics.recall_score(Y_test, Y_dpred, average='weighted', labels=np
.unique(Y_dpred))
```

```
d_f1 = metrics.f1_score(Y_test, Y_dpred, average='weighted', labels=np.unique(
Y_dpred))
print('Decision Tree - Acc:', d_accuracy, 'Prec:', d_precision, 'Rec:', d_reca
ll, 'F1:', d_f1)

# Multinomial NB
MultiNB = MultinomialNB()
MultiNB.fit(X_train, Y_train)
Y_mpred = MultiNB.predict(X_test)
m_accuracy = metrics.accuracy_score(Y_test, Y_mpred)
m_precision = metrics.precision_score(Y_test, Y_mpred, average='weighted', lab
els=np.unique(Y_mpred))
m_recall = metrics.recall_score(Y_test, Y_mpred, average='weighted', labels=np
.unique(Y_mpred))
m_f1 = metrics.f1_score(Y_test, Y_mpred, average='weighted', labels=np.unique(
Y_mpred))
print('MultinomialNB - Acc:', m_accuracy, 'Prec:', m_precision, 'Rec:', m_reca
ll, 'F1:', m_f1)

# KNN
KNN = KNeighborsClassifier()
KNN.fit(X_train, Y_train)
Y_kpred = KNN.predict(X_test)
k_accuracy = metrics.accuracy_score(Y_test, Y_kpred)
k_precision = metrics.precision_score(Y_test, Y_kpred, average='weighted', lab
els=np.unique(Y_kpred))
k_recall = metrics.recall_score(Y_test, Y_kpred, average='weighted', labels=np
.unique(Y_kpred))
k_f1 = metrics.f1_score(Y_test, Y_kpred, average='weighted', labels=np.unique(
Y_kpred))
print('KNN - Acc:', k_accuracy, 'Prec:', k_precision, 'Rec:', k_recall, 'F1:',
k_f1)

# Line graph
plt.figure(figsize=(16,10))
classifiers = ['Bernoulli', 'Random Forest', 'Gaussian', 'Decision Tree', 'Mul
tinomial', 'KNeighbors']
acc = [b_accuracy, r_accuracy, g_accuracy, d_accuracy, m_accuracy, k_accuracy]
prec = [b_precision, r_precision, g_precision, d_precision, m_precision, k_pre
cision]
rec = [b_recall, r_recall, g_recall, d_recall, m_recall, k_recall]
f1 = [b_f1, r_f1, g_f1, d_f1, m_f1, k_f1]
plt.plot(classifiers, acc, marker='o', label='Accuracy')
plt.plot(classifiers, prec, marker='o', label='Precision')
plt.plot(classifiers, rec, marker='o', label='Recall')
plt.plot(classifiers, f1, marker='o', label='F1')
plt.title("Scores of Applied Classifiers")
plt.legend()
plt.show()

# Bar graph
plt.figure(figsize=(16,10))
left = [1,2,3,4,5,6]
height = [b_f1, r_f1, g_f1, d_f1, m_f1, k_f1]
tick_label = ['Bernoulli', 'Random Forest', 'Gaussian', 'Decision Tree', 'Mult
inomial', 'KNeighbors']
plt.bar(left, height, tick_label=tick_label, width=0.9,
        color=['#08737f', '#00898a', '#089f8f', '#39b48e', '#64c987', '#92dc7e
'])
plt.xlabel('Classifiers')
plt.ylabel('F1 Scores')
plt.title('F1 Scores of Applied Classifiers')
plt.show()
```

Lab 12: Flask Introduction

I learned how to deploy a web app with Flask. Using a Tic-Tac-Toe example, I understood the structure (app.py, templates/, static/) and how to serve a frontend.

```
# app.py
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)

# templates/index.html (excerpt)
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Tic Tac Toe</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}
">
</head>
<body>
    <h1>Tic Tac Toe</h1>
    <div id="gameBoard">
        {% for i in range(9) %}
        <div class="cell" onclick="makeMove(this)"></div>
        {% endfor %}
    </div>
    <div id="status"></div>
    <button onclick="resetGame()">Reset Game</button>
    <script src="{{ url_for('static', filename='script.js') }}"></script>
</body>
</html>

# static/style.css (glowing aesthetic)
body {
    text-align: center;
    font-family: Arial, sans-serif;
    background: #0f0f0f;
    color: #fff;
}
h1 {
    margin-top: 20px;
    color: #00ffcc;
    text-shadow: 0 0 10px #00ffcc;
}
#gameBoard {
    display: grid;
    grid-template-columns: repeat(3, 100px);
    grid-gap: 10px;
    justify-content: center;
    margin: 30px auto;
}
.cell {
    width: 100px;
    height: 100px;
    background: #222;
    border: 2px solid #00ffcc;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 36px;
    cursor: pointer;
```

```
    color: #00ffcc;
    box-shadow: 0 0 10px #00ffcc;
}
.cell:hover {
    background: #333;
}
#status {
    font-size: 24px;
    margin: 20px;
    color: #ff66cc;
}
button {
    padding: 10px 20px;
    font-size: 16px;
    background: #ff66cc;
    border: none;
    border-radius: 5px;
    color: #fff;
    cursor: pointer;
    box-shadow: 0 0 10px #ff66cc;
}

# static/script.js (game logic)
let currentPlayer = "X";
let gameActive = true;
let board = ["", "", "", "", "", "", "", "", ""];

function makeMove(cell) {
    const index = Array.from(cell.parentNode.children).indexOf(cell);
    if (!gameActive || board[index]) return;
    board[index] = currentPlayer;
    cell.textContent = currentPlayer;
    if (checkWinner()) {
        document.getElementById("status").textContent = `${currentPlayer} Wins
!`;
        gameActive = false;
    } else if (!board.includes("")) {
        document.getElementById("status").textContent = "It's a Draw!";
        gameActive = false;
    } else {
        currentPlayer = currentPlayer === "X" ? "O" : "X";
    }
}

function checkWinner() {
    const winPatterns = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ];
    return winPatterns.some(pattern =>
        pattern.every(index => board[index] === currentPlayer)
    );
}

function resetGame() {
    board = ["", "", "", "", "", "", "", "", ""];
    currentPlayer = "X";
    gameActive = true;
    document.querySelectorAll(".cell").forEach(cell => cell.textContent = "");
    document.getElementById("status").textContent = "";
}
```

Project: Smart Volunteer Dispatch

For my semester project, I built a **Smart Volunteer Dispatch** system that automatically assigns the right volunteer to a community emergency. When a ticket arrives with a description and location, an NLP model first classifies the emergency type (medical, fire, flood, food distribution, etc.). Then a Prolog knowledge base matches the best available volunteer based on their skills, proximity to the location, and current availability. The whole system runs as a Flask web app where a dispatcher can submit an emergency and instantly see the assigned volunteer.

How It Works

1. NLP Classification: I trained a Multinomial Naïve Bayes classifier on a small dataset of emergency descriptions and their categories. The model and TF-IDF vectorizer are saved and loaded in the app.
2. Logic Rules in Prolog: The file `volunteer_rules.pl` defines facts about volunteers (skills, location, status) and rules to find the best match based on the emergency type and location.
3. Flask Frontend: A clean web page where the user enters the emergency details and gets the assigned volunteer's name, skills, and contact information.

Project Structure

```
volunteer_dispatch/
│
├── app.py                      # Main Flask app
├── train_emergency_model.py    # One-time script to train NLP model
├── volunteer_rules.pl          # Prolog knowledge base for volunteer matching
├── models/
│   ├── emergency_model.pkl
│   └── vectorizer.pkl
├── templates/
│   └── index.html
└── static/
    └── style.css
```

1. Training the NLP Model (train_emergency_model.py)

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
import joblib

# Sample dataset: 'description' and 'category'
data = pd.read_csv('emergency_data.csv')
# Example rows:
# description,category
# "a person fell and is bleeding heavily",medical
# "a small fire in the kitchen",fire
# "water entering houses after heavy rain",flood
# "need warm meals for 50 displaced families",food_distribution
# ...

X = data['description']
y = data['category']

vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
X_vec = vectorizer.fit_transform(X)

model = MultinomialNB()
model.fit(X_vec, y)

joblib.dump(model, 'models/emergency_model.pkl')
joblib.dump(vectorizer, 'models/vectorizer.pkl')
print("Emergency classification model saved.")
```

2. Prolog Volunteer Rules (volunteer_rules.pl)

```
% volunteer_rules.pl
% Volunteer facts: id, name, skill_list, location, is_available
volunteer(1, 'Ali', [medical, first_aid], 'BlockA', 1).
volunteer(2, 'Sara', [fire_safety, rescue], 'BlockB', 1).
volunteer(3, 'Ahmed', [food_prep, logistics], 'BlockC', 0).
volunteer(4, 'Fatima', [medical, shelter], 'BlockA', 1).
volunteer(5, 'Bilal', [flood_rescue, driving], 'BlockD', 1).

% Skill matching based on emergency type
required_skill(medical, medical).
required_skill(medical, first_aid).
required_skill(fire, fire_safety).
required_skill(flood, flood_rescue).
required_skill(food_distribution, food_prep).
required_skill(food_distribution, logistics).

% Proximity: same block is considered near
near_location(Loc1, Loc2) :- Loc1 == Loc2.

% Rule to assign the best available volunteer
assign_volunteer(EmergencyType, Location, VolunteerId, VolunteerName) :-
    required_skill(EmergencyType, Skill),
    volunteer(VolunteerId, VolunteerName, Skills, VolLocation, 1),
    member(Skill, Skills),
    near_location(Location, VolLocation).

% Helper: member/2
member(X, [X|_]).
member(X, [_|T]) :- member(X, T).
```

3. Flask Application (app.py)

```
from flask import Flask, render_template, request
import joblib
from pyswip import Prolog

app = Flask(__name__)

# Load NLP model
model = joblib.load('models/emergency_model.pkl')
vectorizer = joblib.load('models/vectorizer.pkl')

# Connect to Prolog
prolog = Prolog()
prolog.consult('volunteer_rules.pl')

def assign_volunteer(emergency_type, location):
    """Use Prolog to find the most suitable volunteer."""
    query = f"assign_volunteer({emergency_type}, {location}, Id, Name)"
    try:
        result = list(prolog.query(query))
        if result:
            # Return the first match (for simplicity; can be improved with sco
ring)
            return result[0]['Id'], result[0]['Name']
        else:
            return None, None
    except Exception as e:
        print(f"Prolog error: {e}")
        return None, None

@app.route('/', methods=['GET', 'POST'])
def index():
    assignment = None
    if request.method == 'POST':
        description = request.form['description']
        location = request.form['location']

        # NLP classification
        X_vec = vectorizer.transform([description])
        predicted_category = model.predict(X_vec)[0]

        # Prolog volunteer assignment
        vol_id, vol_name = assign_volunteer(predicted_category, location)

        assignment = {
            'category': predicted_category,
            'vol_id': vol_id,
            'vol_name': vol_name if vol_name else 'No available volunteer foun
d',
            'description': description,
            'location': location
        }
    return render_template('index.html', result=assignment)

if __name__ == '__main__':
    app.run(debug=True)
```

4. Frontend (templates/index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart Volunteer Dispatch</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}
">
</head>
<body>
    <div class="container">
        <h1> Smart Volunteer Dispatch</h1>
        <p>Report an emergency and let the system assign the right volunteer.<
/p>
        <form method="post">
            <label>Emergency Description:</label><br>
            <input type="text" name="description" placeholder="e.g., a person
needs medical help" required>
            <br><label>Location (Block):</label><br>
            <input type="text" name="location" placeholder="BlockA" required>
            <br>
            <button type="submit">Dispatch Volunteer</button>
        </form>

        {% if result %}
        <div class="result">
            <h2>Dispatch Result</h2>
            <p><strong>Emergency Category:</strong> {{ result.category }}</p>
            <p><strong>Assigned Volunteer:</strong> {{ result.vol_name }}</p>
            {% if result.vol_id %}
            <p><strong>Volunteer ID:</strong> {{ result.vol_id }}</p>
            {% endif %}
            <p><em>Description:</em> {{ result.description }}</p>
            <p><em>Location:</em> {{ result.location }}</p>
        </div>
        {% endif %}
    </div>
</body>
</html>
```

5. Styling (static/style.css)

```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: #eef2f5;
    margin: 0;
    padding: 2rem;
    display: flex;
    justify-content: center;
}
.container {
    background: white;
    padding: 2rem;
    border-radius: 10px;
    max-width: 600px;
    width: 100%;
    box-shadow: 0 0 15px rgba(0,0,0,0.1);
}
input[type="text"] {
    width: 80%;
    padding: 0.6rem;
    margin: 0.5rem 0 1rem;
    border: 1px solid #ccc;
    border-radius: 5px;
}
button {
    padding: 0.7rem 1.4rem;
    background: #2c3e50;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}
button:hover {
    background: #1a252f;
}
.result {
    margin-top: 2rem;
    padding: 1rem;
    background: #f0f8ff;
    border-left: 4px solid #2c3e50;
}
```

How to Run the Project

1. Create `emergency_data.csv` with columns `description` and `category`.
2. Run `python train_emergency_model.py` to build the NLP model.
3. Start Flask: `python app.py`.
4. Open `http://127.0.0.1:5000`, submit an emergency description, and see the assigned volunteer.

— All labs and project completed —